

幻尔科技

机械臂高级控制手册



Hiwonder 官方网站



版本号	修改日期	修改摘要
V1.0	2025/08/09	初次发布

目录

1. 齐次坐标变换	5
1.1 齐次坐标简介	5
1.2 齐次坐标作用	6
1.3 变换	6
1.3.1 平移变换	6
1.3.2 比例变换	7
1.3.3 对称变换	8
1.3.4 旋转变换	10
2. DH 建模	12
2.1 DH 建模介绍	12
2.1.1 DH 建模简介	12
2.1.2 DH 建模作用	12
2.1.3 DH 建模步骤	12
2.2 ArmPi Ultra 的 DH 建模	20
3. 正运动学分析	22
3.1 正运动学介绍	22
3.1.1 正运动学简介	22
3.1.2 正运动学基本原理	22
3.2 ArmPi Ultra 正运动分析	23
3.3 正运动学服务介绍	24

3.3.1 准备工作	24
3.3.2 查看运动学服务	24
3.4 通过服务控制舵机	27
3.4.1 通过服务调用	27
3.4.2 启动 launch 文件控制	27
3.4.4 开启 APP 自启动服务	28
3.5 程序分析	28
3.5.1 launch 文件分析	28
3.5.2 python 文件分析	29
4. 逆运动学分析	32
4.1 逆运动学介绍	32
4.1.1 逆运动学简介	32
4.1.2 逆运动学简析	32
4.2 ArmPi Ultra 逆运动分析	33
4.3 逆运动学服务介绍	34
4.3.1 准备工作	35
4.3.2 查看运动学服务	35
4.4 通过服务控制舵机	37
4.4.1 通过服务调用	37
4.4.2 启动 launch 文件控制	38
4.4.4 开启 APP 自启动服务	38
4.5 程序分析	39

4.5.1 launch 文件分析	39
4.5.2 python 文件分析	39

机械臂的高级控制

1. 齐次坐标变换

1.1 齐次坐标简介

齐次坐标变换是一种数学工具，广泛应用于计算机图形学、机器人学、计算机视觉等领域，用于描述坐标系之间的变换关系。它的主要目的是简化复杂的几何变换，如平移、旋转、缩放和投影等，使其可以以一种更简洁的方式进行表示和计算。

二维平面内，我们用一对坐标值 (x, y) 来表示一个点在平面内的确切位置。假如变换前的点的坐标为 (x, y) ，变换后的点坐标为 (x^*, y^*) ，这个变换过程可以写成如下矩阵形式：

$$\begin{aligned} [x^*, y^*] &= [x, y] \bullet M \\ \begin{cases} x^* = a_1x + b_1y + c_1 \\ y^* = a_2x + b_2y + c_2 \end{cases} \\ [x^*, y^*] &= [x \quad y \quad 1] \begin{bmatrix} a_1 & a_2 \\ b_1 & b_2 \\ c_1 & c_2 \end{bmatrix} \end{aligned}$$

这种用三维向量表示二维向量，或者一般而言，用一个 $n+1$ 维的向量表示一个 n 维向量的方法称为齐次坐标表示法。

普通坐标与齐次坐标的关系为“一对多”：

普通坐标 $\times h \rightarrow$ 齐次坐标

齐次坐标 $\div h \rightarrow$ 普通坐标

当 $h = 1$ 时产生的齐次坐标称为“规格化坐标”，因为前 n 个坐标就是普通坐标系下的 n 维坐标

1.2 齐次坐标作用

采用了齐次坐标表示法，就可以统一的把二维线性变换表示为如下所示的规格化形式：

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} a_1 & a_2 & 0 \\ b_1 & b_2 & 0 \\ c_1 & c_2 & 1 \end{bmatrix}$$

对于一个图形，可以用顶点表来描述图形的几何关系，用连边表来描述图形的拓扑关系。

所以对图形的变化，最后转变成，只是要变化图形的顶点表。

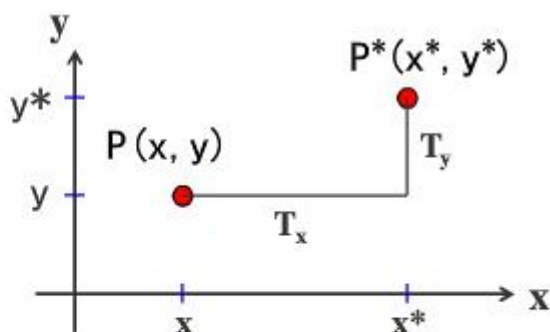
1.3 变换

1.3.1 平移变换

平移是指将 P 点沿直线路径从一个坐标位置移到另一个坐标位置的重定位过程。

即新的坐标分别在x方向和y方向增加了一个增量和，使得：

$$\begin{cases} x^* = x + T_x \\ y^* = y + T_y \end{cases}$$



T_x , T_y 称为**平移矢量**

齐次坐标计算形式如下：

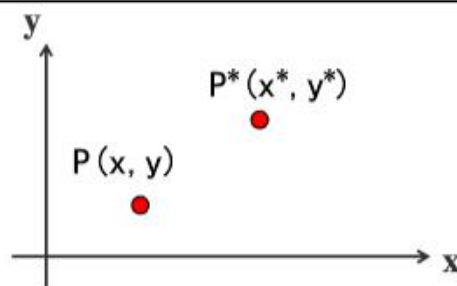
$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ T_x & T_y & 1 \end{bmatrix} = \begin{bmatrix} x + T_x & y + T_y & 1 \end{bmatrix}$$

平移是一种不产生变形而移动物体的刚体变换，即物体上的每个点移动相同数量的坐标。

1.3.2 比例变换

比例变换是指对p点相对于坐标原点沿x方向放缩 S_x 倍，沿y方向放缩 S_y 倍。其中 S_x 和 S_y 称为比例系数

$$\begin{cases} x^* = x \bullet S_x \\ y^* = y \bullet S_y \end{cases}$$

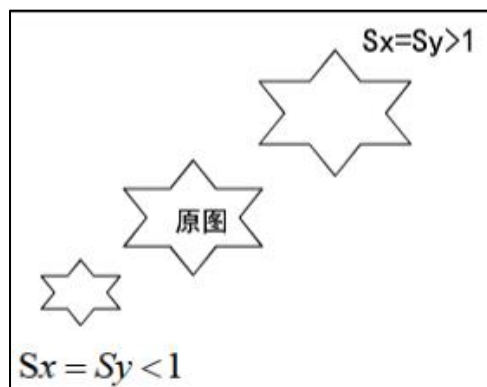


齐次坐标计算形式如下：

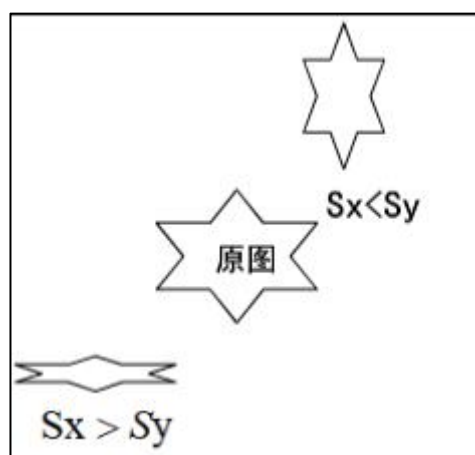
$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \bullet \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} S_x \bullet x & S_y \bullet y & 1 \end{bmatrix}$$

缩放系数 S_x , S_y 可赋予任何正整数。值小于 1, 则会缩小物体的尺寸。值大于 1, 则会放大物体。都指定为 1, 物体尺寸就不会改变。

(1) $S_x = S_y$ 比例



(2) $S_x \neq S_y$ 比例

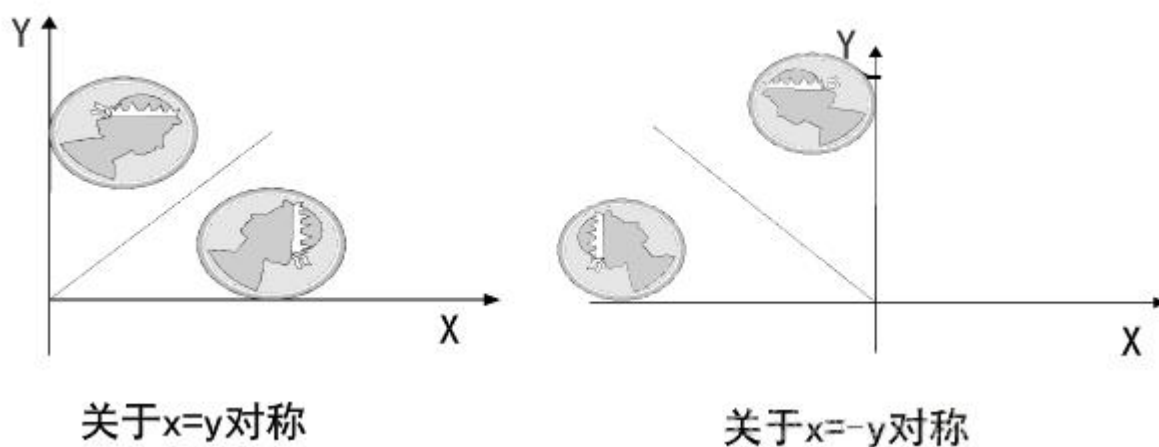
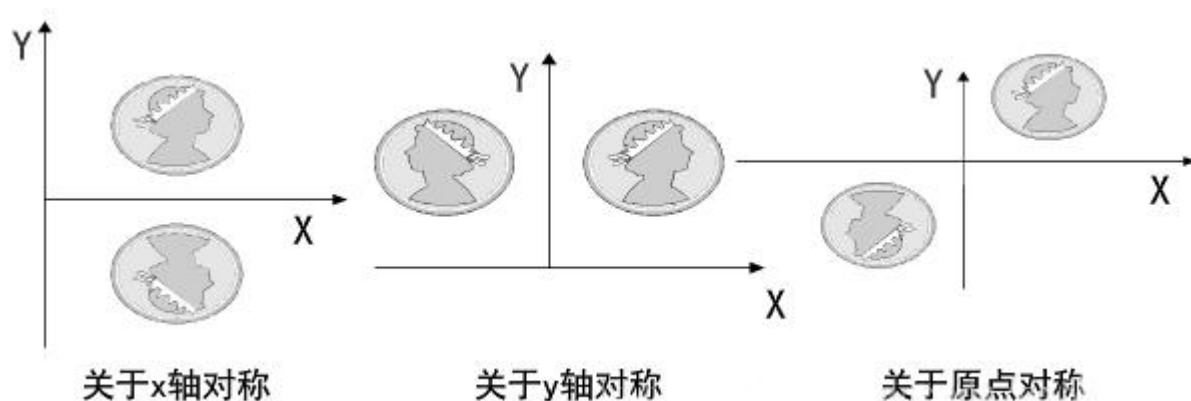


当 $S_x = S_y$ 时, 比例变换成为整体比例变换, 用以下矩阵进行计算:

$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & S \end{bmatrix} = \begin{bmatrix} x & y & S \end{bmatrix} = \begin{bmatrix} \frac{x}{S} & \frac{y}{S} & 1 \end{bmatrix}$$

1.3.3 对称变换

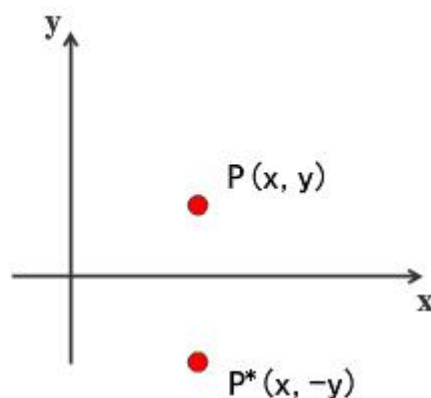
对称变换也称为反射变化或镜像变换, 变换后的图形是原图形关于某一轴线或原点的镜像。



关于 x 轴对称

(a) 关于X轴对称

点P经过关于X轴的对称变换后形成点P*，则 $x^*=x$ 且 $y^*=-y$ ，写成齐次坐标的计算形式为：

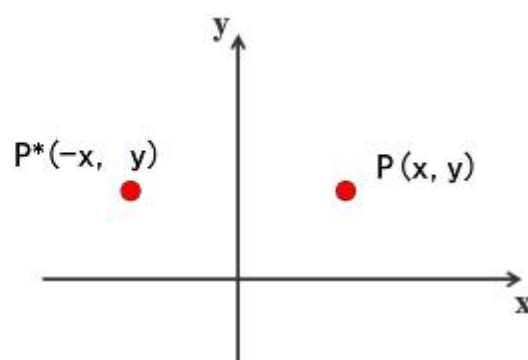


$$[x^* \quad y^* \quad 1] = [x \quad y \quad 1] \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [x \quad -y \quad 1]$$

关于 y 轴对称

(b) 关于y轴对称

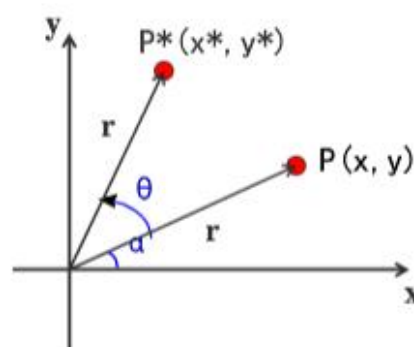
点P经过关于X轴的对称变换后形成点P*, 则 $x^*=-x$ 且 $y^*=y$, 写成齐次坐标的计算形式为:



$$\begin{bmatrix} x^* & y^* & 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} -x & y & 1 \end{bmatrix}$$

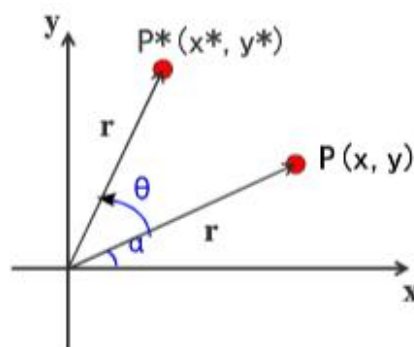
1.3.4 旋转变换

二维旋转是指将P点绕坐标原点转动某个角度 θ (逆时针为正, 顺时针为负) 得到新的点P*的重定位过程



首先确定当基准点为坐标原点时, 点位置P旋转的变换方程

应用标准三角特性, 利用角度 α 和 θ 将转换后的坐标表示为:



$$\begin{cases} x^* = r \cos(\alpha + \theta) = r \cos \alpha \cos \theta - r \sin \alpha \sin \theta \\ y^* = r \sin(\alpha + \theta) = r \cos \alpha \sin \theta + r \sin \alpha \cos \theta \end{cases}$$

在极坐标系中点的原始坐标为：

$$\begin{cases} x = r \cos \alpha \\ y = r \sin \alpha \end{cases}$$

将x、y 代入上式，就得到相对于原点将在位置 (x, y) 处的点旋转 θ 角的变换方程：

$$\begin{cases} x^* = x \cos \theta - y \sin \theta \\ y^* = x \sin \theta + y \cos \theta \end{cases}$$

二维图形绕原点逆时针旋转 θ 角的齐次坐标计算形式可写为：

$$\begin{aligned} [x^* \quad y^* \quad 1] &= [x \quad y \quad 1] \bullet \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &= [x \bullet \cos \theta - y \bullet \sin \theta \quad x \bullet \sin \theta + y \bullet \cos \theta \quad 1] \end{aligned}$$

绕原点顺时针旋转 θ 角的齐次坐标计算形式可写为：

$$[x^* \quad y^* \quad 1] = [x \quad y \quad 1] \bullet \begin{bmatrix} \cos(-\theta) & \sin(-\theta) & 0 \\ -\sin(-\theta) & \cos(-\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

2. DH 建模

2.1 DH 建模介绍

2.1.1 DH 建模简介

DH 建模方法一种基于齐次变换矩阵的低副机构建模方法。常用于描述机械臂关节之间几何关系和运动学的方法。它是机械工程领域中用于建立机械臂运动学模型的标准方法之一。DH 建模通过一组参数来描述机械臂的各个关节之间的相对位置和运动，从而实现正向运动学和逆向运动学的计算。

DH 建模的基本思想是将机械臂的每个关节看作一个旋转或平移的连续刚体链接，然后通过一系列的转动和平移来模拟整个机械臂的运动。

2.1.2 DH 建模作用

DH 建模主要用在机器人运动学上，这种方法在每个连杆上建立一个坐标系，通过齐次坐标变换来实现两个连杆上坐标的变换，在多连杆串联的系统中，多次使用齐次坐标变换，就可以建立首末坐标系的关系。

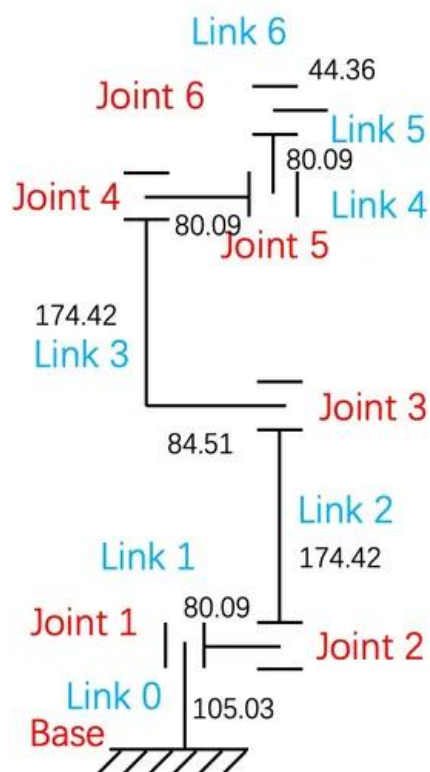
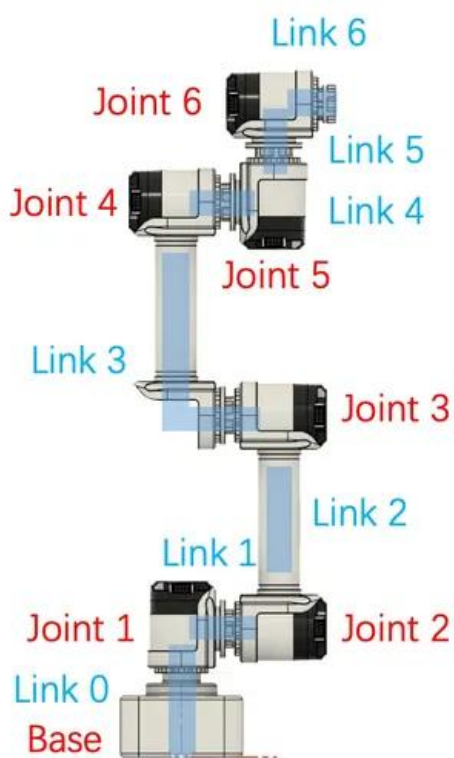
2.1.3 DH 建模步骤

列表

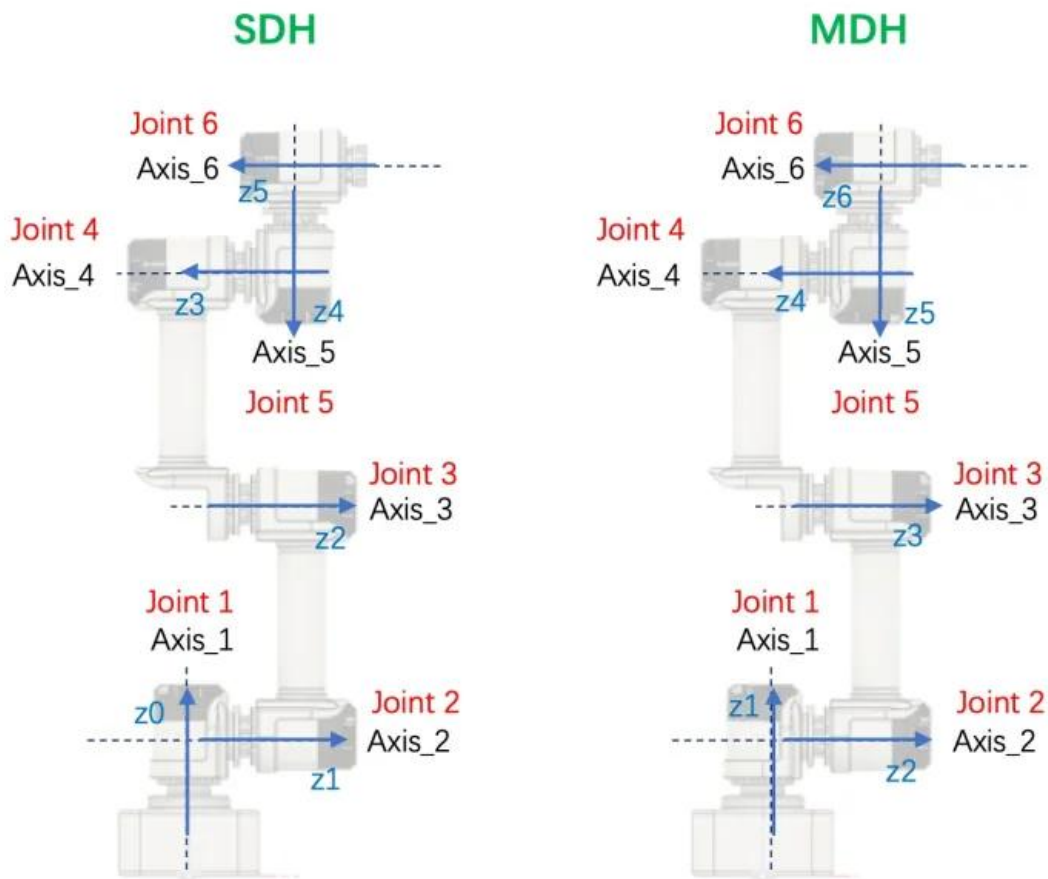
SDH						MDH					
i	theta _i	d _i	a _i	alpha _i	offset	i	theta _i	d _i	a _{i-1}	alpha _{i-1}	offset
1	q1					1	q1				
2	q2					2	q2				
3	q3					3	q3				
4	q4					4	q4				
5	q5					5	q5				
6	q6					6	q6				

建模

1) 确定初始位置并给连杆和关节标号。一般机械臂初始位置被认为是整体伸直朝上，横向关节朝向为 x 轴。下图右边是连杆结构图。



2) 确认各关节轴线和 z 轴。轴线为各关节旋转或移动的轴线。z 轴为当前轴线，方向任意（此处选择电机旋转反方向）。此处两种方法将有不同，要注意下角标的符号。z 轴的下标就代表了第几坐标系了。



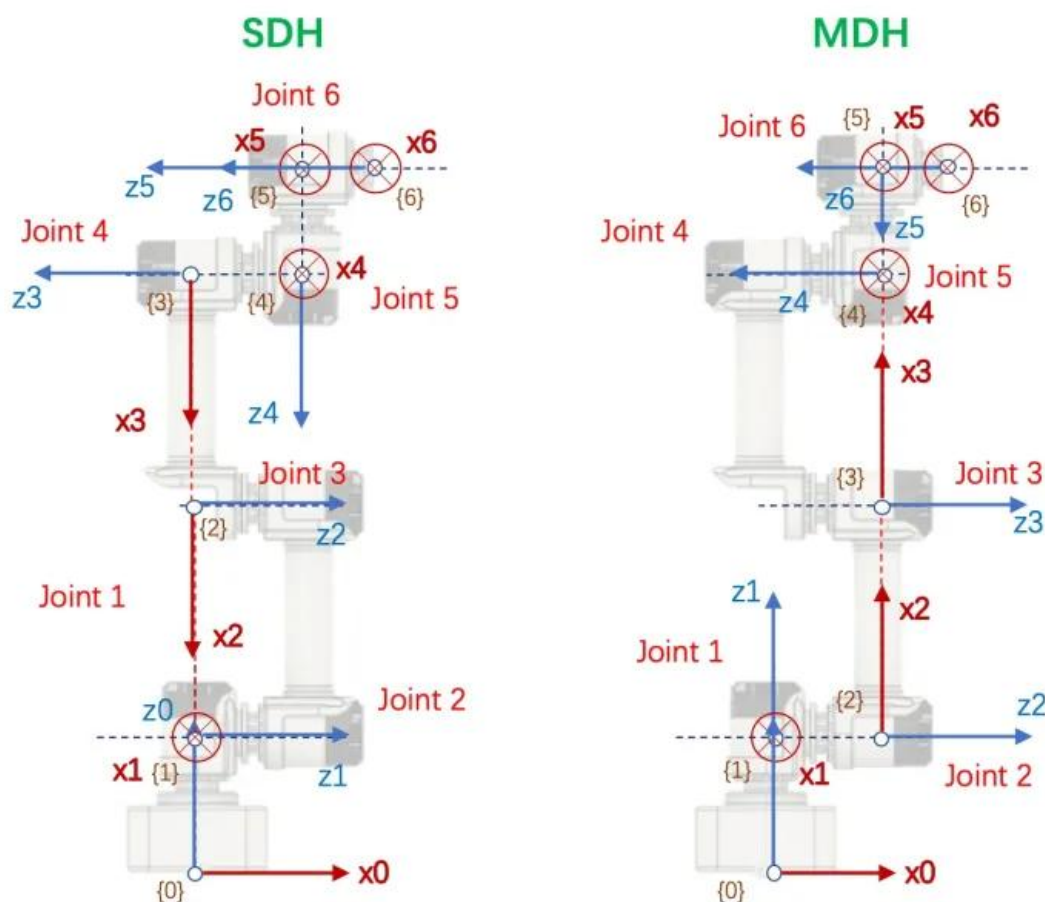
建立各关节坐标系

首先找第 i 个坐标系 $\{i\}$ 的原点。再确定 x_i 轴。在 SDH 中， $\{i\}$ 的轴线和 $\{i-1\}$ 的轴线，如果相交，则交点为原点， x_i 轴在两轴线所在平面的垂线上，方向任意；如果不相交，两轴线公垂线和当前关节 i 轴线的交点为原点， x_i 为在公垂线上，方向任意。在 MDH 中， $\{i\}$ 的轴线和 $\{i+1\}$ 的轴线，如果相交，则交点为原点， x_i 轴在两轴线所在平面的垂线上，方向任意；如果不相交，两轴线公垂线和当前关节 i 轴线的交点为原点， x_i 为在公垂线上，方向任意。由于会有两轴线平行，公垂线有无数条； x 轴 z 轴方向可任意设置，所以会导致参数模型会有很多版本。

添加首尾坐标系

SDH 中已经标注好 $\{0\}$ 了，所以在尾端添加 $\{6\}$ 。MDH 中已经标注好 $\{6\}$ 了，所以在首端添加 $\{0\}$ 。首尾端的坐标系设置是要根据实际情况和个人喜好的。

下面是建系过程：



注意:由于选择 z 轴, x 轴和原点有很大的自由性, 所以以上只是建系的一种选择, 怎么建系需要根据自身情况来判断。

确定 D-H 的 4 个参数

拿出第一步列的表, 挨个进行填写。先不要在意参数下角标, 因为会混淆。SDH 和 MDH 两种方法的坐标变换都是从前面的坐标系往下一个坐标系变换, 写第 i 行时只要关注 $\{i-1\}$ 和 $\{i\}$ 之间的关系就好。但是参数顺序是有差别的。

此处将对表的第 2 行进行举例。 $i=2$ 时, 利用 $\{1\}$ 的坐标系变换到 $\{2\}$ 。

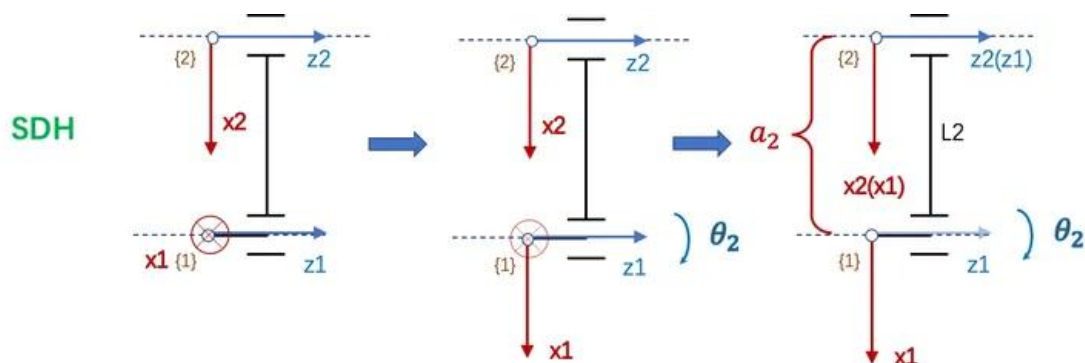
SDH 的顺序是 $\theta \rightarrow d \rightarrow \alpha \rightarrow a$ 。

1) 先将坐标系 $i-1$ 的 x_{i-1} 轴绕 z_i 轴转到与坐标系 i 的 x_i 轴平行, 转过的角度为 θ_i , 右手准则拇指指向为 z 正方向为正;

2) 再将 x_{i-1} 轴沿 z_{i-1} 轴平移到 x_i 轴, 平移的距离为 d_i 。

3) 接着刚才变换的结果, 将坐标系 $i-1$ 的 z_{i-1} 轴绕 x_{i-1} 轴转到和坐标系的 z_i 轴平行, 转过的角度为 α_i , 右手准则拇指指向为 x 正方向为正;

4) 再将 z_{i-1} 轴沿 x_{i-1} 轴平移到 z_i 轴, 平移的距离为 a_i , 直到两坐标系重合。



根据 SDH 的规则可以得出下表。在上图 $\theta_2 = -90^\circ$, 但由于是可变参数 θ 设为 q_2 。

i	$\theta_{i/^\circ}$	d_i/mm	a_i/mm	$\alpha_{i/^\circ}$
2	q_2	0	-174.42	0

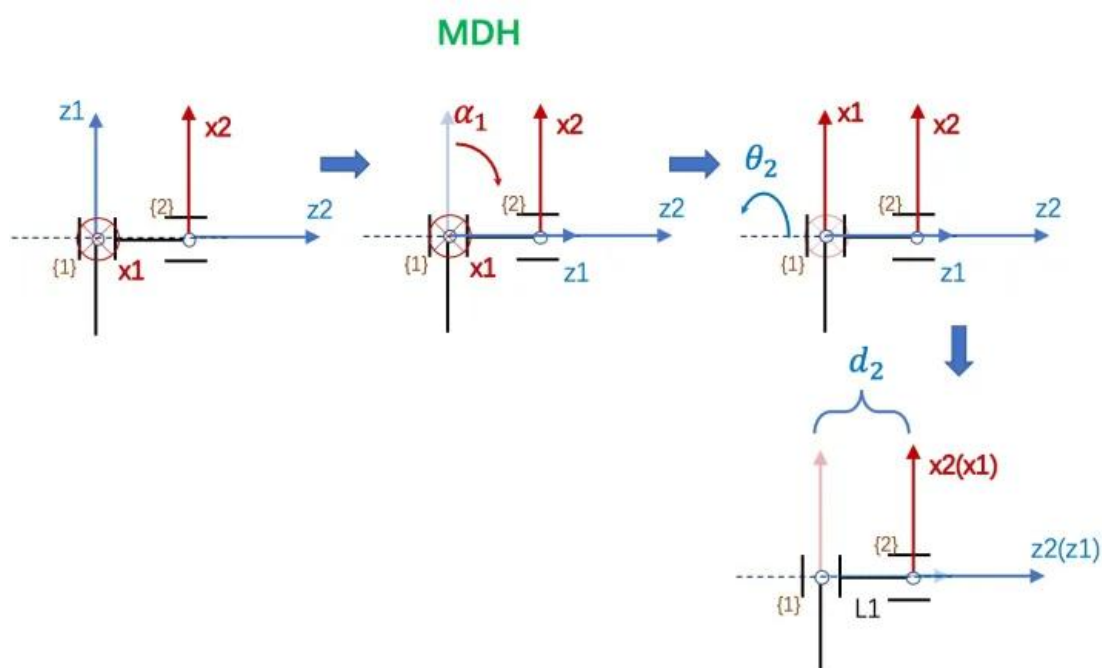
MDH 的顺序是 $\alpha \rightarrow a \rightarrow \theta \rightarrow d$ 。

1) 先将坐标系 $i-1$ 的 z_{i-1} 轴绕 x_i 轴转到与坐标系 i 的 z_i 轴平行, 转过的角度为 α_{i-1} , 右手准则拇指指向为 z 正方向为正;

2) 再将 z_{i-1} 轴沿 x_{i-1} 轴平移到 z_i 轴, 平移的距离为 a_{i-1} 。

3) 接着刚才变换的结果, 将坐标系 $i-1$ 的 x_{i-1} 轴绕 z_{i-1} 轴转到和坐标系的 x_i 轴平行, 转过的角度为 θ_i , 右手准则拇指指向为 x 正方向为正;

4) 再将 x_{i-1} 轴沿 z_{i-1} 轴平移到 x_i 轴, 平移的距离为 d_i , 直到两坐标系重合。



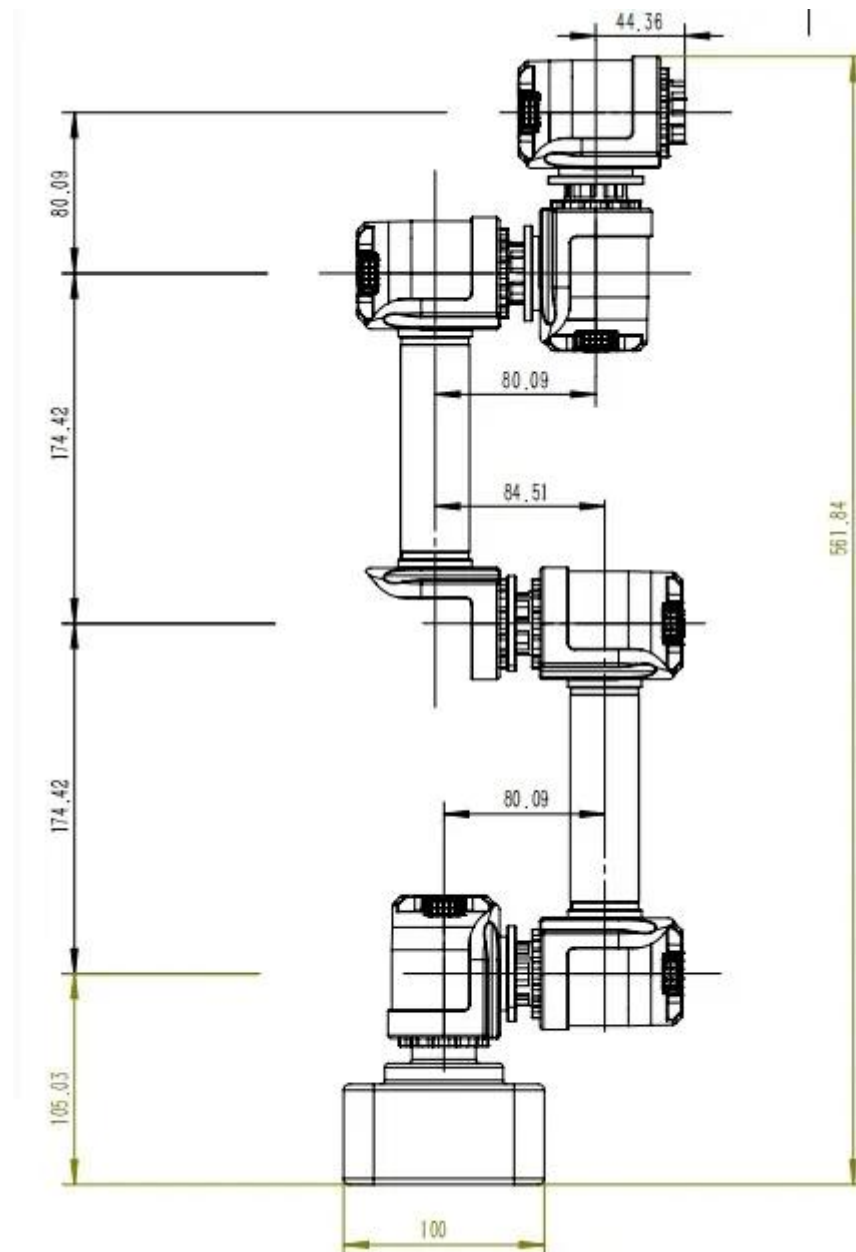
根据 MDH 的规则可以得出下表。在上图 $\theta_2=90^\circ$ ，但由于是可变参数 θ 设为 q_2 。

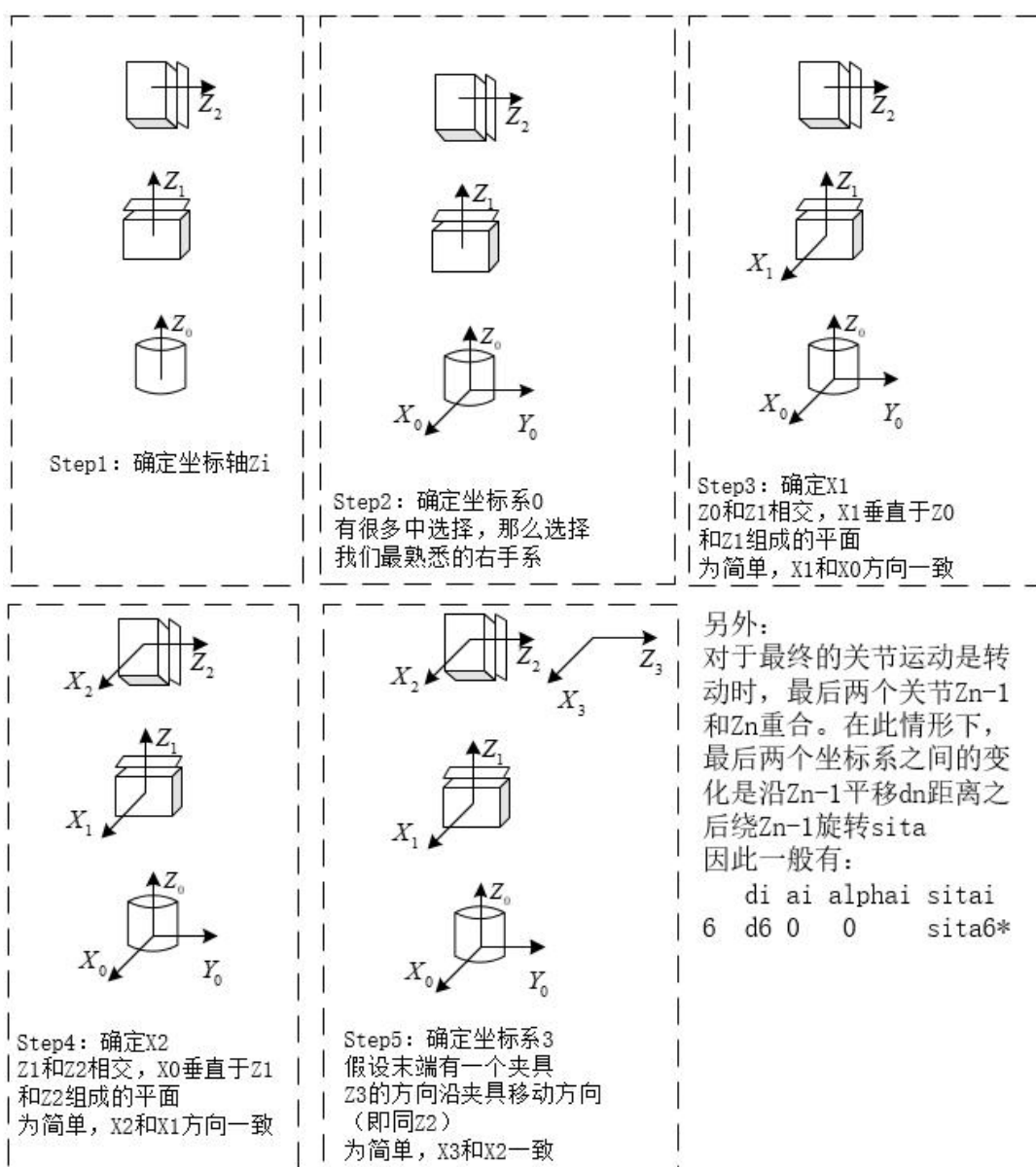
i	$\theta_{i-1}/^\circ$	d_i/mm	a_{i-1}/mm	$\alpha_{i-1}/^\circ$
2	q_2	80.09	0	90

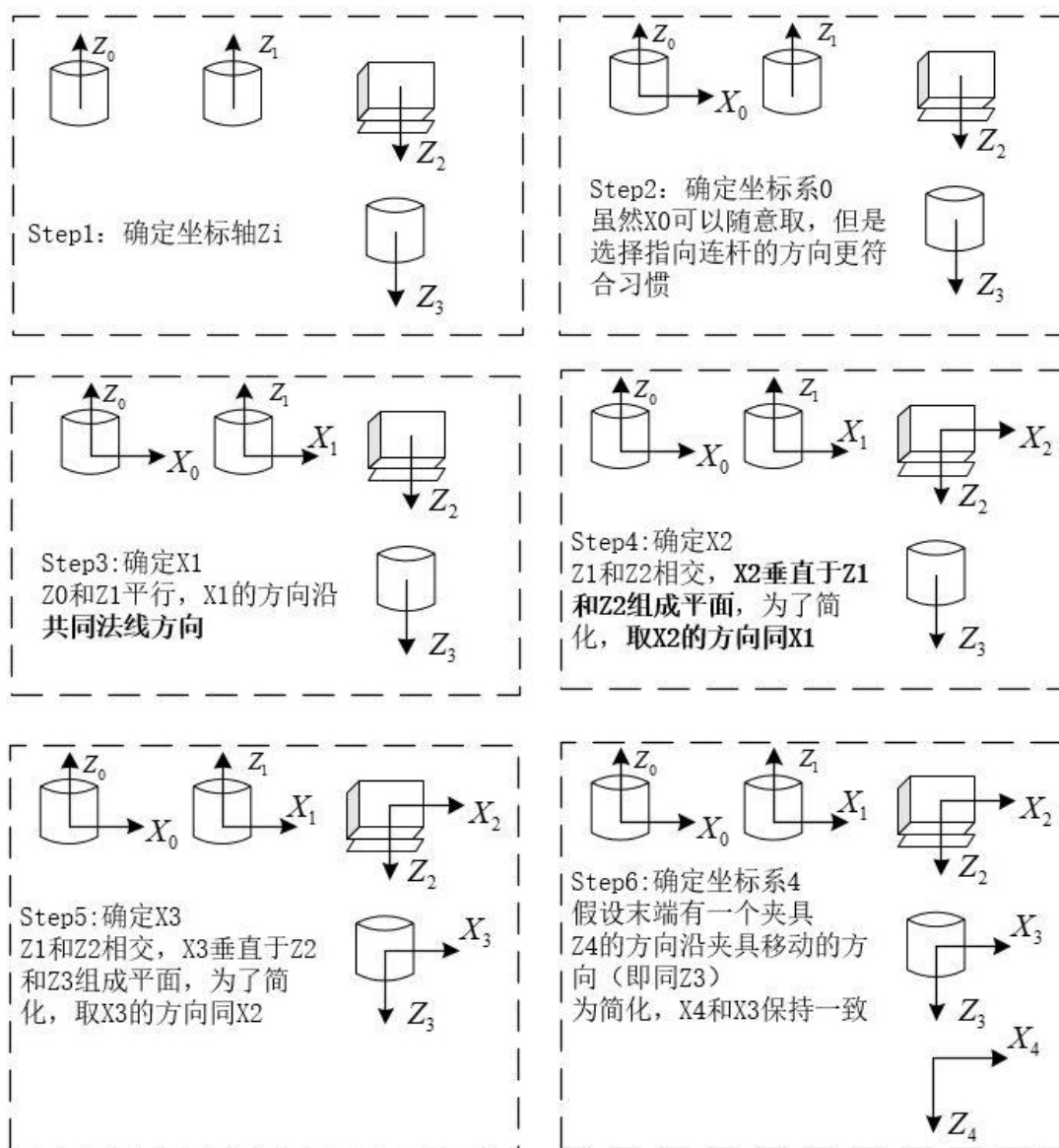
完整表

根据 DH 法模型和工程图纸，将表填好。（官方给的二维图纸和三维模型尺寸不一致。由于主要使用三维模型，所以自己重新量了一下三维图纸）

SDH						MDH					
i	θ_{i-1}	d_i	a_i	α_{i-1}	offset	i	θ_{i-1}	d_i	a_{i-1}	α_{i-1}	offset
1	q_1	105.03	0	$\pi/2$	$\pi/2$	1	q_1	105.03	0	0	$\pi/2$
2	q_2	0	-174.42	0	$-\pi/2$	2	q_2	80.09	0	$\pi/2$	$\pi/2$
3	q_3	0	-174.42	π	0	3	q_3	0	174.42	0	0
4	q_4	-75.66	0	$-\pi/2$	$-\pi/2$	4	q_4	4.44	174.42	$\pi/2$	$\pi/2$
5	q_5	-80.09	0	$\pi/2$	0	5	q_5	-80.09	0	$-\pi/2$	0
6	q_6	-44.36	0	0	0	6	q_6	-44.36	0	$\pi/2$	0

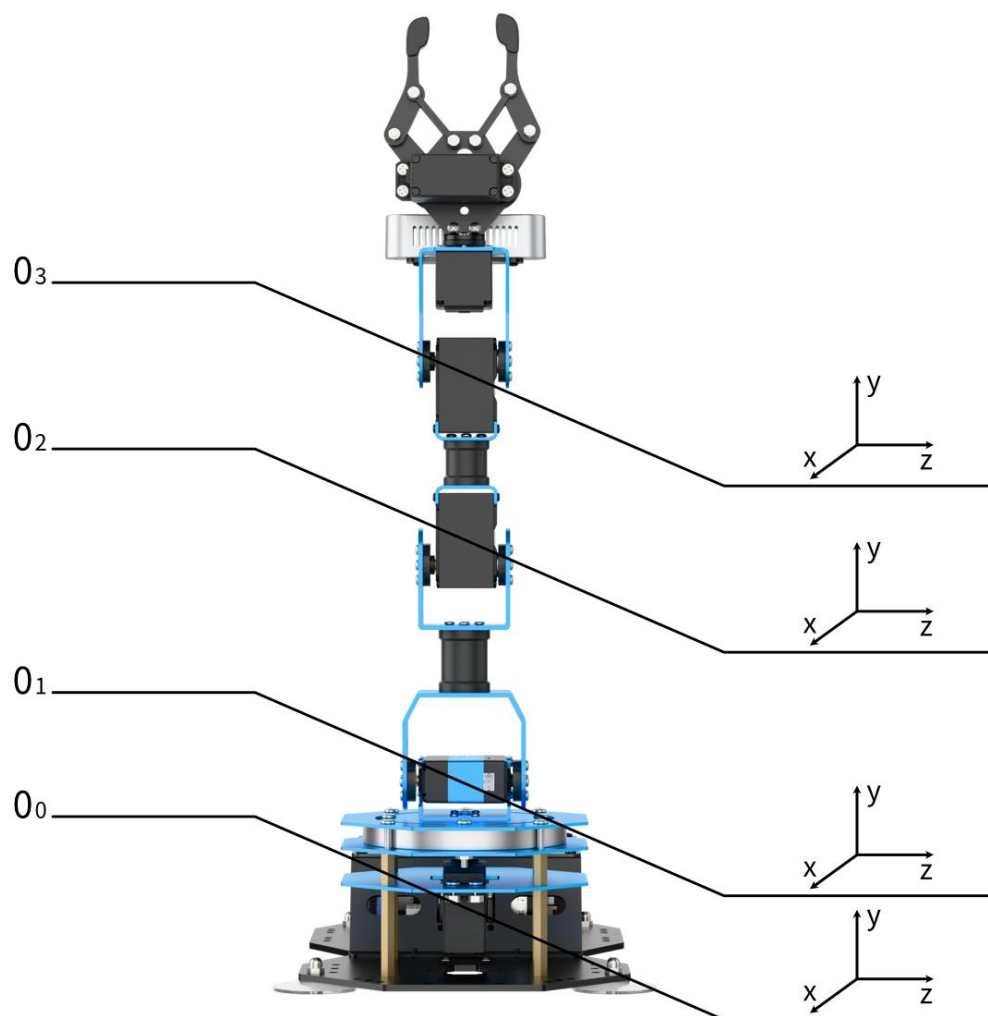






2.2 ArmPi Ultra 的 DH 建模

ArmPi Ultra DH 坐标系:



DH 建模列表:

可参考文件:

“/home/ubuntu/ros2_ws/src/driver/kinematics/kinematics/transform.py”

i	α_{i-1}	a_{i-1}	d_i	θ_i	θ 范围
1	θ	θ	θ	θ_1	$\theta_1(-120, 120)$
2	-90	θ	θ	θ_2	$\theta_2(-180, 0)$

3	0	0.10048	0	03	03(-120, 120)
4	0	0.100	0	04	04(-200, 20)
5	-90	0	0	05	05(-120, 120)

3. 正运动学分析

3.1 正运动学介绍

3.1.1 正运动学简介

正向运动学是指从机器人的关节运动推导出末端执行器的运动的过程,也就是从机器人的关节坐标计算出末端执行器的位置和姿态信息的过程。

正向运动学更容易理解和实现,因为它是直接从机器人关节的运动来计算末端执行器的位置和姿态。在机器人控制中,正向运动学被广泛用于轨迹规划、仿真和可视化等方面。

3.1.2 正运动学基本原理

机械臂的正运动学通常表述为如下的函数形式:

$$\xi_E = \mathcal{K}(q)$$

它表明末端执行器的位姿是基于关节坐标的一个函数。若用齐次变换,其表达式将由(2)式所给的单个连杆变换矩阵的简单乘积。对于 N 个机械臂则有:

$$\xi_E \sim {}^0T_E = {}^0A_1 {}^1A_2 \cdots {}^{N-1}A_N$$

对于任何一个机械臂,无论其关节的数量和关节如何,都可以计算出其正向运动学解。通常真正实用的机械臂的任务空间是三维的,控制机械臂运动,就是要给机械臂每个关节一

个特定的电压控制信号，使他们各自运动一个特定的角度，得到期望的位姿状态，对于一个 6 轴机械臂，其总的矩阵变换通常被写作 T_6 。

3.2 ArmPi Ultra 正运动分析

i	α_{i-1}	a_{i-1}	d_i	θ_i	θ 范围
1	0	0	0	θ_1	$\theta_1(-120, 120)$
2	-90	0	0	θ_2	$\theta_2(-180, 0)$
3	0	0.10048	0	θ_3	$\theta_3(-120, 120)$
4	0	0.100	0	θ_4	$\theta_4(-200, 20)$
5	-90	0	0	θ_5	$\theta_5(-120, 120)$

DH 的齐次变换矩阵形式：

$${}^{i-1}_iT = T_{\hat{X}_{i-1}}(\alpha_{i-1})T_{\hat{X}_R}(\alpha_{i-1})T_{\hat{Z}_Q}(\theta_i)T_{\hat{Z}_P}(d_i)$$

$$= \begin{bmatrix} c\theta_i & -s\theta_i & 0 & a_{i-1} \\ s\theta_i c\alpha_{i-1} & c\theta_i c\alpha_{i-1} & -s\alpha_{i-1} & -s\alpha_{i-1}d_i \\ s\theta_i s\alpha_{i-1} & c\theta_i s\alpha_{i-1} & c\alpha_{i-1} & c\alpha_{i-1}d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

得到 ArmPi Ultra 单个关节的齐次变换矩阵：

$$T(0-1) = \begin{bmatrix} c\theta_1 & -s\theta_1 & 0 & 0 \\ s\theta_1 & c\theta_1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T(1-2) = \begin{bmatrix} c\theta_2 & -s\theta_2 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -s\theta_2 & -c\theta_2 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T(2-3) = \begin{bmatrix} c\theta_3 & -s\theta_3 & 0 & a_2 \\ s\theta_3 & c\theta_3 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T(3-4) = \begin{bmatrix} C\theta_4 & -S\theta_4 & 0 & a_3 \\ S\theta_4 & C\theta_4 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$T(4-5) = \begin{bmatrix} C\theta_5 & -S\theta_5 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ -S\theta_5 & -C\theta_5 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

将单个关节的齐次坐标相乘得到 ArmPi Ultra 正运动学公式：

$$T(0-5) = T(0-1) * T(1-2) * T(2-3) * T(3-4) * T(4-5)$$

由于机械臂上的 5、4、3 号舵机，都是水平旋转， $T(1-2)$ 、 $T(2-3)$ 、 $T(3-4)$ 可通过积化和差公式化简：

$$\theta_2 + \theta_3 + \theta_4 = 0 \text{ (水平旋转时，总转角抵消为 } 0 \text{)}$$

$$\cos(\theta_2 + \theta_3 + \theta_4) = 1$$

$$\sin(\theta_2 + \theta_3 + \theta_4) = 0$$

根据机械臂跟关节的几何关系可得：

$$T(1-4) = \begin{bmatrix} 1 & 0 & 0 & a_2 C\theta_2 + a_3 C\theta_{23} \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & -a_2 S\theta_2 - a_3 S\theta_{23} \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

得到机械臂运动学公式如下：

$$T(0-5) = \begin{bmatrix} C\theta_1 C\theta_5 - S\theta_1 S\theta_5 & -C\theta_1 S\theta_5 - S\theta_1 C\theta_5 & 0 & C\theta_1 (C\theta_2 a_2 + C\theta_{23} a_3) \\ S\theta_1 C\theta_5 + C\theta_1 S\theta_5 & -S\theta_1 S\theta_5 + C\theta_1 C\theta_5 & 1 & S\theta_1 (C\theta_2 a_2 + C\theta_{23} a_3) \\ 0 & 0 & -1 & -S\theta_2 a_2 - S\theta_{23} a_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

3.3 正运动学服务介绍

3.3.1 准备工作

- 1) 根据 “1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意

事项”内容安装好机械臂平台。

2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。

3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

3.3.2 查看运动学服务

1) 单击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

2) 输入命令启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

3) 打开一个新的命令栏 ，输入命令查看机械臂的运动学服务。

```
ros2 service list
```

```
/kinematics/describe_parameters  
/kinematics/get_current_pose  
/kinematics/get_joint_range  
/kinematics/get_link  
/kinematics/get_parameter_types  
/kinematics/get_parameters  
/kinematics/init_finish  
/kinematics/list_parameters  
/kinematics/set_joint_range  
/kinematics/set_joint_value_target  
/kinematics/set_link  
/kinematics/set_parameters  
/kinematics/set_parameters_atomically  
/kinematics/set_pose_target  
/kinematics/set_pose_target_smooth
```

服务	功能说明
/kinematics/describe_parameters	描述参数
/kinematics/get_current_pose	获取当前机器人的位姿信息
/kinematics/get_joint_range	获取机器人各个关节的运动范围
/kinematics/get_link	获取机器人的关节链接信息
/kinematics/get_parameter_types	获取参数类型
/kinematics/get_parameters	获取参数
/kinematics/init_finish	初始化完成
/kinematics/list_parameters	列出参数
/kinematics/set_joint_range	设置机器人关节的运动范围
/kinematics/set_joint_value_target	设置机器人的目标关节角度值，以实现姿

	态控制
/kinematics/set_link	设置机器人的关节链接信息
/kinematics/set_parameters	设置参数
/kinematics/set_parameters_atomically	原子性的设置参数
/kinematics/set_pose_target	设置机器人末端执行器的目标姿态
/kinematics/set_pose_target_smooth	平滑设置机器人末端执行器目标姿态

3.4 通过服务控制舵机

3.4.1 通过服务调用

1) 打开一个新的命令栏 ，输入命令。

```
ros2 service call /kinematics/set_joint_value_target kinematics_msgs/srv/SetJointValue "{joint_value: [500.0, 400.0, 300.0, 400.0, 500.0 ]}"
```

```
> ros2 service call /kinematics/set_joint_value_target kinematics_msgs/srv/SetJointValue "{joint_value: [500.0, 400.0, 300.0, 400.0, 500.0 ]}"
```

2) 将 1-5 号舵机的旋转角度输入，即可得到机械臂移动后的位姿信息，这里是以 xyz 三轴的坐标和用四元数表示方向。注意：这里的服务调用只有位姿信息的显示，机械臂并不会移动。

```
requester: making request: kinematics_msgs.srv.SetJointValue_Request(joint_value=[500.0, 400.0, 300.0, 400.0, 500.0])
response:
kinematics_msgs.srv.SetJointValue_Response(success=True, solution=True, pose=geometry_msgs.msg.Pose(position=geometry_msgs.msg.Point(x=0.0001952335886763884, y=0.0, z=0.4477525933481886), orientation=geometry_msgs.msg.Quaternion(x=0.0, y=-0.7071067811865475, z=0.0, w=0.7071067811865475)))
```

3.4.2 启动 launch 文件控制

1) 在启动机械臂 SDK 文件和 kinematics 文件的窗口下，按下“Ctrl+C”关闭启动

文件。

2) 接着再输入命令开启正运动学服务调用程序, 可以看到机械臂根据设置好的舵机脉冲宽度计算并移动到目标位置, 终端可以看到相关信息。

```
ros2 launch example fk.launch.py
```

```
> ros2 launch example fk.launch.py
```

```
[fk-5] [INFO] [1753884915.748940529] [fk_demo]: 舵机脉宽值: [500, 500, 500, 120, 500]
[fk-5] [INFO] [1753884915.749498418] [fk_demo]: 舵机脉宽值转为弧度: (0.0, -1.57079632679
48966, 0.0, 0.020943951023932005, 0.0)
[fk-5] [INFO] [1753884915.750851844] [fk_demo]: 正运动学解-坐标: [0.16996271619072376, 0
.0, 0.2915247886198293]
[fk-5] [INFO] [1753884915.751532270] [fk_demo]: 正运动学解-四元数: geometry_msgs.msg.Qua
ternion(x=-0.0, y=0.010471784116245848, z=-0.0, w=0.999945169365512)
[fk-5] [INFO] [1753884915.752360436] [fk_demo]: 转换四元数成rpy: (-0.0, 1.20000000000000
62, -0.0)
```

3) 若要关闭玩法, 按下“Ctrl+C”即可, 若玩法关闭失败, 可重复多次按下。

3.4.4 开启 APP 自启动服务

1) 玩法体验完成后, 需开启 APP 服务 (若不开启则会影响后续 APP 玩法的使用)。

在终端界面  输入命令, 按下回车, 启动 APP 服务, 接着再稍等片刻即可开启。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

2) 启动完成后机械臂会回到初始姿态, 蜂鸣器也会“滴”一声。

3.5 程序分析

3.5.1 launch 文件分析

launch 文件位于:

```
/home/ubuntu/ros2_ws/src/example/example/simple/fk.launch.py
```

◆ generate_launch_description 函数

```
def generate_launch_description():
    compiled = os.environ['need_compile']

    if compiled == 'True':
        sdk_package_path = get_package_share_directory('sdk')
    else:
        sdk_package_path = '/home/ubuntu/ros2_ws/src/driver/sdk'

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(sdk_package_path, 'launch/arm_pi_ultra.launch.py')
        ]),
    )

    fk_node = Node(
        package='example',
        executable='fk',
        output='screen',
    )

    return LaunchDescription([
        sdk_launch,
        fk_node
    ])
```

包含 sdk 功能包中的 launch/arm_pi_ultra.launch.py 启动文件，用于启动 SDK 相关节点（如机械臂控制底层服务）。定义 fk_node 节点，启动 example 功能包中的 fk 可执行文件，输出日志到屏幕，用于演示机械臂正运动学求解功能。

◆ 主程序

```
if __name__ == '__main__':
    # 创建一个LaunchDescription对象
    ld = generate_launch_description()

    ls = LaunchService()
    ls.include_launch_description(ld)
    ls.run()
```

创建 LaunchDescription 对象和 LaunchService 服务，将启动描述添加到启动服务并运行，实现启动整个系统的功能。

3.5.2 python 文件分析

python 文件位置位于

/home/ubuntu/ros2_ws/src/example/example/simple/include/fk.py

◆ 导入功能包

```
import rclpy
from rclpy.node import Node
import signal
from std_srvs.srv import Trigger
import kinematics.transform as transform
from kinematics.forward_kinematics import ForwardKinematics
from servo_controller_msgs.msg import ServoPosition, ServosPosition
```

- ① **kinematics.transform**: 一个自定义模块（假设定义在 **kinematics** 包内），包含一些与运动学相关的函数，如脉宽与角度的转换、四元数与欧拉角的转换等。
- ② **kinematics.forward_kinematics**: 另一个自定义模块，定义了 **ForwardKinematics** 类，用于计算机械臂的正运动学解。
- ③ **servo_controller_msgs.msg**: 包含控制舵机位置的消息类型 **ServoPosition** 和 **ServosPosition**。

◆ 初始化节点

```
class FkDemo(Node):
    def __init__(self):
        super().__init__('fk_demo') # 初始化节点
        self.fk = ForwardKinematics(debug=True)

        # 创建总线舵机发布者
        self.servos_pub = self.create_publisher(ServosPosition, '/servo_controller', 1)

        # 等待机械臂底层控制服务启动
        self.client = self.create_client(Trigger, '/controller_manager/init_finish')
        self.client.wait_for_service()
        self.client = self.create_client(Trigger, '/kinematics/init_finish')
        self.client.wait_for_service()

        # 输出舵机脉宽值和正运动学解

        self.servo_list = [500, 500, 500, 120, 500]
        self.duration = 1.0
        self.get_logger().info("舵机脉宽值: {}".format(self.servo_list))

        pulse = transform.pulse2angle([i for i in self.servo_list]) # 舵机脉宽值转为弧度
        self.get_logger().info("舵机脉宽值转为弧度: {}".format(pulse))

        res = self.fk.get_fk(pulse) # 获取运动学正解
        self.get_logger().info('正运动学解-坐标: {}'.format(res[0]))
        self.get_logger().info('正运动学解-四元数: {}'.format(res[1]))
        self.get_logger().info('转换四元数成rpy: {}'.format(transform.qua2rpy(res[1])))

        #self.set_servo_position(self.servos_pub, self.duration, self.servo_list)#发布舵机信息，驱动机械臂
```

初始化 ROS 节点(名称为 fk_demo),创建正运动学求解器实例(ForwardKinematics); 创建舵机控制发布者 (话题为 /servo_controller) ; 等待机械臂控制器 (/controller_manager/init_finish) 和运动学服务 (/kinematics/init_finish) 启动, 确保底层控制就绪。定义初始舵机脉宽值列表 (servo_list) 和运动持续时间 (duration) , 输出舵机脉宽值日志。将舵机脉宽值转换为弧度 (通过 transform.pulse2angle) , 输出转换结果。调用正运动学求解器 (fk.get_fk) 计算末端执行器位姿, 输出坐标、四元数及转换后的 RPY 角 (滚转、俯仰、偏航角) 。

◆ set_servo_position 函数

```
def set_servo_position(self, pub, duration, positions):
    msg = ServosPosition()
    msg.duration = float(duration)
    position_list = []
    for i in range(1, 6):
        position = ServoPosition()
        position.id = i
        position.position = float(positions[i-1])
        position_list.append(position)
    msg.position = position_list
    msg.position_unit = "pulse"

    pub.publish(msg)
```

构造舵机位置消息 (ServosPosition) , 设置运动持续时间 (duration) 和 1-5 号舵机的目标脉宽值; 指定位置单位为 “pulse” (脉宽) , 发布消息到舵机控制话题。

◆ 主函数

主函数启动节点。

```
def main(args=None):
    rclpy.init(args=args) # 初始化 rclpy
    fk_demo_node = FkDemo() # 创建节点
    rclpy.spin(fk_demo_node) # 保持节点运行
    fk_demo_node.destroy_node() # 销毁节点
    rclpy.shutdown() # 关闭 rclpy
```

初始化 ROS 2, 创建 FkDemo 节点实例。通过 rclpy.spin 保持节点运行。程序结束时

销毁节点并关闭 ROS2。

4. 逆运动学分析

4.1 逆运动学介绍

4.1.1 逆运动学简介

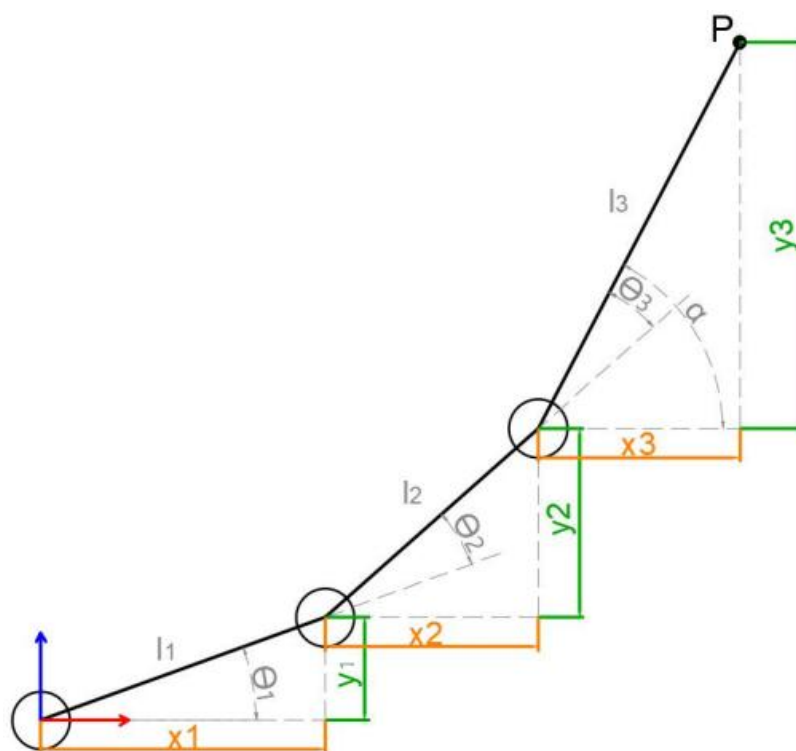
逆运动学是决定要达成所需要的姿势所要设置的关节可活动对象的参数的过程。

机械臂的逆运动学问题是其轨迹规划与控制的重要基础, 逆运动学求解是否快速准确将直接影响到机械臂轨迹规划与控制的精度, 因此针对六自由度机械臂, 设计一种快速准确的逆运动学求解方法是十分重要的。

4.1.2 逆运动学简析

对于机械臂而言, 就是给出夹持器的位置和朝向后求出每个关节的旋转角度。机械臂的三维运动是比较复杂的, 这里为了简化模型, 我们去掉下方云台的旋转关节, 这样就可以在二维的平面上进行运动学分析了。

进行逆运动学分析一般要进行大量的矩阵运算, 过程复杂计算量大所以实现起来较难。为了更好地适应我们的需要, 我们使用几何法对机械臂进行分析。



我们将机械臂的模型简化，去掉底座云台，和执行器部分得到机械臂的主体。从上图可以看到机械臂的端点 P 的坐标 (x, y) ，最终由三个部分组成 $(x_1 + x_2 + x_3, y_1 + y_2 + y_3)$ 。

其中上图的 $\theta_1, \theta_2, \theta_3$ 就是我们要求解的舵机的角度， α 是爪子与水平面的夹角。从图上来看显然爪子的俯视角 $\alpha = \theta_1 + \theta_2 + \theta_3$ ，据此我们可以列出下式：

$$x = l_1 \cos \theta_1 + l_2 \cos (\theta_1 + \theta_2) + l_3 \cos (\theta_1 + \theta_2 + \theta_3)$$

$$y = l_1 \sin \theta_1 + l_2 \sin (\theta_1 + \theta_2) + l_3 \sin (\theta_1 + \theta_2 + \theta_3)$$

其中 x, y 由使用者给出， l_1, l_2, l_3 为机械臂的机械结构固有属性。

我们可求出三个舵机的角度，然后根据角度控制舵机即可实现坐标位置的控制。

4.2 ArmPi Ultra 逆运动分析

定义点 W 为第三根连杆的起点（即第二关节的位置）。由于第三根连杆长度为 l_3 ，方向角为 α ，因此：

$$x_w = x - l_3 \cos \alpha, \quad y_w = y - l_3 \sin \alpha$$

点 W 坐标由前两个连杆决定：

$$x_w = l_1 \cos \theta_1 + l_2 \cos(\theta_1 + \theta_2)$$

$$y_w = l_1 \sin \theta_1 + l_2 \sin(\theta_1 + \theta_2)$$

$$d = \sqrt{x_w^2 + y_w^2}$$

可得：

$$\cos \theta_2 = \frac{d^2 - l_1^2 - l_2^2}{2l_1 l_2}$$

$$\theta_2 = \pm \arccos \left(\frac{d^2 - l_1^2 - l_2^2}{2l_1 l_2} \right)$$

定义 ϕ 为点 W 的方向角：

$$\phi = \arctan 2(y_w, x_w)$$

$$\beta = \arctan 2(l_2 \sin \theta_2, l_1 + l_2 \cos \theta_2)$$

可得：

$$\theta_1 = \phi - \beta$$

$$\theta_3 = \alpha - \theta_1 - \theta_2$$

4.3 逆运动学服务介绍

4.3.1 准备工作

1) 根据“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->配件安装及注意事项”内容安装好机械臂平台。

2) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->机器人开机说明”将机械臂开机。

3) 可参照“1 ArmPi Ultra 使用入门\ArmPi Ultra 使用入门手册->开发环境搭建及配置”进入机械臂系统桌面。

4.3.2 查看运动学服务

1) 单击 ，启动命令栏，输入指令，按下回车，关闭自启动服务。

```
~/ .stop_ros.sh
```

```
> ~/ .stop_ros.sh
```

2) 输入命令启动机械臂 SDK 文件。

```
ros2 launch sdk armpi_ultra.launch.py
```

```
> ros2 launch sdk armpi_ultra.launch.py
```

3) 打开一个新的命令栏 ，输入命令查看机械臂的运动学服务。

```
ros2 service list
```

```
/kinematics/describe_parameters  
/kinematics/get_current_pose  
/kinematics/get_joint_range  
/kinematics/get_link  
/kinematics/get_parameter_types  
/kinematics/get_parameters  
/kinematics/init_finish  
/kinematics/list_parameters  
/kinematics/set_joint_range  
/kinematics/set_joint_value_target  
/kinematics/set_link  
/kinematics/set_parameters  
/kinematics/set_parameters_atomically  
/kinematics/set_pose_target  
/kinematics/set_pose_target_smooth
```

服务	功能说明
/kinematics/describe_parameters	描述参数
/kinematics/get_current_pose	获取当前机器人的位姿信息
/kinematics/get_joint_range	获取机器人各个关节的运动范围
/kinematics/get_link	获取机器人的关节链接信息
/kinematics/get_parameter_types	获取参数类型
/kinematics/get_parameters	获取参数
/kinematics/init_finish	初始化完成
/kinematics/list_parameters	列出参数
/kinematics/set_joint_range	设置机器人关节的运动范围
/kinematics/set_joint_value_target	设置机器人的目标关节角度值，以实现姿

	态控制
/kinematics/set_link	设置机器人的关节链接信息
/kinematics/set_parameters	设置参数
/kinematics/set_parameters_atomically	原子性地设置参数
/kinematics/set_pose_target	设置机器人末端执行器的目标姿态
/kinematics/set_pose_target_smooth	平滑设置机器人末端执行器目标姿态

4.4 通过服务控制舵机

4.4.1 通过服务调用

1) 打开一个新的命令栏，输入命令，可以使用“Tab”键进行补齐。

```
ros2 service call /kinematics/set_pose_target kinematics_msgs/srv/SetRobotPose "{position: [0.3, 0.0, 0.2], pitch_range: [-180,180], pitch: 10, resolution: 1 }"
```

```
> ros2 service call /kinematics/set_pose_target kinematics_msgs/srv/SetRobotPose "{position: [0.3, 0.0, 0.2], pitch_range: [-180,180], pitch: 10, resolution: 1 }"
```

2) 输入目标位置的 xyz 三轴坐标（必须为浮点类型），pitch 为舵机的俯仰角，取值范围为 $(-180, 180)$ ，pitch_range 为角度范围，取值范围为 $(-180, 180)$ ，resolution 为调节角度范围，固定取 1.0 就行。

注意：如果设置的目标位置和角度机械臂无法成功到达则终端无法显示对应舵机信息。这里的服务调用只有位姿信息的显示，机械臂并不会移动。

```
waiting for service to become available...
requester: making request: kinematics_msgs.srv.SetRobotPose_Request(position=[0.3
, 0.0, 0.2], pitch=10.0, pitch_range=[-180.0, 180.0], resolution=1.0, duration=0.
0)

response:
kinematics_msgs.srv.SetRobotPose_Response(success=True, pulse=[500, 395, 661, 349
, 504], current_pulse=[700, 500, 500, 500, 500], rpy=[1.0, 10.0, 0.0], min_variat
ion=621.0)
```

4.4.2 启动 launch 文件控制

- 1) 在启动机械臂 SDK 文件和 kinematics 文件的窗口下，按下“Ctrl+C”关闭启动文件。
- 2) 接着再输入命令开启正运动学服务调用程序，可以看到机械臂根据设置好的舵机脉冲宽度计算并移动到目标位置，终端可以看到相关信息。

```
ros2 launch example ik.launch.py
```

```
> ros2 launch example ik.launch.py
```

```
[ros_robot_controller-1] [INFO] [1753887617.266177474] [ros_robot_controller]: start
[servo_controller-2] [INFO] [1753887617.463141641] [servo_manager]: start
[servo_controller-2] [INFO] [1753887617.599194141] [controller_manager]: start
[search_kinematics_solutions-4] [INFO] [1753887618.027190881] [kinematics]: start
[ik-5] [INFO] [1753887618.085842270] [ik_demo_node]: 末端坐标: [0.16996271619072376,
0.0, 0.2915247886198293]
[ik-5] [INFO] [1753887618.086729048] [ik_demo_node]: rpy1: [0.0, 1.0, 0.0]
[ik-5] [INFO] [1753887618.087451881] [ik_demo_node]: 舵机脉宽值1: [500.0, 518.3271128
152718, 536.7423694088675, 139.24858992692907, 500.0]
[ik-5] [INFO] [1753887618.088079622] [ik_demo_node]: 舵机脉宽值2: [500.0, 481.6728879
624485, 463.25763059113245, 102.41807596201727, 500.0]
```

- 3) 若要关闭玩法，按下“Ctrl+C”即可，若玩法关闭失败，可重复多次按下。

4.4.4 开启 APP 自启动服务

- 1) 玩法体验完成后，需开启 APP 服务（**若不开启则会影响后续 APP 玩法的使用**）。

在终端界面  输入命令按下回车，启动 APP 服务，接着再稍等片刻即可开启。

```
ros2 launch bringup bringup.launch.py
```

```
> ros2 launch bringup bringup.launch.py
```

- 2) 启动完成后机械臂会回到初始姿态，蜂鸣器也会“滴”一声。

4.5 程序分析

4.5.1 launch 文件分析

launch 文件位于：

`/home/ubuntu/ros2_ws/src/example/example/simple/ik.launch.py`

```
def generate_launch_description():

    compiled = os.environ['need_compile']

    if compiled == 'True':
        sdk_package_path = get_package_share_directory('sdk')
    else:
        sdk_package_path = '/home/ubuntu/ros2_ws/src/driver/sdk'

    sdk_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(sdk_package_path, 'launch/armapi_ultra.launch.py')
        ]),
    )

    ik_node = Node(
        package='example',
        executable='ik',
        output='screen',
    )

    return LaunchDescription([
        sdk_launch,
        ik_node
    ])
```

包含 sdk 功能包中的 `launch/armapi_ultra.launch.py` 启动文件，用于启动 SDK 相关节点（如机械臂控制底层服务、运动学计算服务等）。定义 `ik_node` 节点，启动 `example` 功能包中的 `ik` 可执行文件，输出日志到屏幕，用于演示机械臂逆运动学求解功能。

◆ 主程序

```
if __name__ == '__main__':
    # 创建一个LaunchDescription对象
    ld = generate_launch_description()

    ls = LaunchService()
    ls.include_launch_description(ld)
    ls.run()
```

创建 `LaunchDescription` 对象和 `LaunchService` 服务，将启动描述添加到启动服务

并运行，实现启动整个系统的功能。

4.5.2 python 文件分析

python 文件位置位于：/home/ubuntu/ros2_ws/src/example/example/simple/include/ik.py

◆ 导入功能包

```
import time
import rclpy
from rclpy.node import Node
from std_srvs.srv import Trigger
import kinematics.transform as transform
from kinematics.forward_kinematics import ForwardKinematics
from kinematics.inverse_kinematics import get_ik
from servo_controller_msgs.msg import ServoPosition, ServosPosition
```

① **消息类型**：ServoPosition 和 ServosPosition 是自定义的消息类型，表示舵机的位置和多个舵机的位置信息。

② **运动学模块**：kinematics.transform 和 kinematics.forward_kinematics 引用了一个自定义的运动学模块，这些模块用于处理机械臂的运动学计算（例如前向和逆向运动学）。

◆ 初始化节点

```
def __init__(self):
    super().__init__('ik_demo_node')

    self.coordinate = [0.16996271619072376, 0.0, 0.2915247886198293]
    self.servo_list = []
    self.duration = 1.0

    # 发布总线舵机话题
    self.servos_pub = self.create_publisher(ServosPosition, '/servo_controller', 1)

    #等待机械臂底层控制服务启动
    self.client = self.create_client(Trigger, '/controller_manager/init_finish')
    self.client.wait_for_service()
    self.client = self.create_client(Trigger, '/kinematics/init_finish')
    self.client.wait_for_service()
```

初始化 ROS 节点（名称为 ik_demo_node），定义末端执行器目标坐标（coordinate）、舵机位置列表（servo_list）和运动持续时间（duration）；创建舵机控制发布者（话题为/servo_controller）；等待机械臂控制器（/controller_manager/init_finish）和运动学服务（/kinematics/init_finish）启动，确保底层控制就绪。

```
# 获取逆解
self.get_logger().info(f"末端坐标: {self.coordinate}")
res = get_ik(self.coordinate, 90, [-180, 180])
if res != []:
    for i in range(len(res)):
        self.get_logger().info(f'rpy{i + 1}: {res[i][1]}')
        pulse = transform.angle2pulse(res[i][0])
        self.servo_list = pulse[0]
        for j in range(len(pulse)):
            self.get_logger().info(f'舵机脉宽值{j + 1}: {pulse[j]}')
        pass
    # self.set_servo_position(self.servos_pub, self.duration, self.servo_list)#发布舵机信息，驱动机械臂
else:
    self.get_logger().info("没有逆运动学解，请检查末端坐标")
```

输出末端目标坐标日志，调用逆运动学求解函数（get_ik）计算关节角度（输入参数为坐标、姿态角 90 度及关节范围）。若存在逆解，输出各关节的 RPY 角（姿态角），将关节角度转换为舵机脉宽值（通过 transform.angle2pulse），并输出每个舵机的脉宽值。若不存在逆解，输出提示日志。

◆ set_servo_position 函数

```
def set_servo_position(self, pub, duration, positions):
    msg = ServosPosition()
    msg.duration = float(duration)
    position_list = []
    for i in range(1, 6):
        position = ServoPosition()
        position.id = i
        position.position = float(positions[i-1])
        position_list.append(position)
    msg.position = position_list
    msg.position_unit = "pulse"
    pub.publish(msg)
```

该函数将舵机位置（脉宽值）打包到 ServosPosition 消息中，并通过 self.servos_pub 发布到话题 /servo_controller，从而驱动机械臂的舵机。每个舵机的位置由 ServoPosition 消息中的 id 和 position 属性指定。

◆ main 函数

主函数启动节点。

```
def main(args=None):  
    rclpy.init(args=args) # 初始化 rclpy  
    ik_demo_node = IkDemo() # 创建节点  
    rclpy.spin(ik_demo_node) # 保持节点运行  
    ik_demo_node.destroy_node() # 销毁节点  
    rclpy.shutdown() # 关闭 rclpy
```

初始化 ROS 2，创建 IkDemo 节点实例。通过 rclpy.spin 保持节点运行。程序结束时销毁节点并关闭 ROS2。